# AdonisJS + Edge Student Management CRUD Guide (With Validator)

**Goal:** Build a simple exam-friendly Student Management System using **AdonisJS**, **Lucid ORM**, **Edge templates**, **Validator**, and **Fake Data Seeder**.

This guide is written step by step so you can easily copy commands and code.

---

## 0. What We Are Building

Features:

```
1. Add student
2. Show all students
3. Edit student
4. Update student
5. Delete student
6. Validate student input
7. Add fake student data using Factory and Seeder
```

Main formula:

```
Migration -> Model -> Validator -> Factory/Seeder -> Controller -> Route -> Edge View
```

Meanings:

```
Migration  = creates database table
Model      = connects code with database table
Validator  = checks form/input data before saving
Factory    = creates fake data objects
Seeder     = inserts fake data into database
Controller = contains logic
Route      = URL path
Edge View  = HTML UI page
```

---

## 1. Create AdonisJS Project

### 1.1 Check Node and npm

```
node -v
npm -v
```

### 1.2 Create project

```
npm create adonisjs@latest student-api
```

When it asks project options, choose a simple/default project. If it asks for database, choose **SQLite** for easiest setup.

Go inside the project:

```
cd student-api
```

### 1.3 Run server

```
npm run dev
```

Open browser:

```
http://localhost:3333
```

If something opens, project is working.

---

## 2. Add Edge Template Engine

Edge is the AdonisJS template engine. It lets us create HTML pages inside AdonisJS.

```
node ace add edge
```

If Edge is already installed, no problem.

---

## 3. Add Lucid Database Package if Needed

If your project already supports migration, skip this step.

Check:

```
node ace make:migration test_check
```

If this command works, delete the test migration file and continue.

If the command does not work, run:

```
node ace add lucid
```

Choose **SQLite** when asked.

---

## 4. Create Students Migration

Migration creates the database table.

```
node ace make:migration students
```

A file will be created inside:

`database/migrations/`

Open the new migration file and paste this code:

```
import { BaseSchema } from '@adonisjs/lucid/schema'

export default class extends BaseSchema {
  protected tableName = 'students'

  async up() {
    this.schema.createTable(this.tableName, (table) => {
      table.increments('id')
      table.string('name').notNullable()
      table.string('email').notNullable()
      table.string('phone').nullable()
      table.string('department').notNullable()
      table.integer('semester').notNullable()
      table.timestamp('created_at', { useTz: true })
      table.timestamp('updated_at', { useTz: true })
    })
  }

  async down() {
    this.schema.dropTable(this.tableName)
  }
}
```

Run migration:

```
node ace migration:run
```

Now the `students` table is created.

If you made a mistake in migration:

```
node ace migration:rollback
node ace migration:run
```

---

## 5. Create Student Model

Model connects AdonisJS code with the database table.

```
node ace make:model Student
```

Open:

`app/models/student.ts`

Paste this code:

```ts
import { DateTime } from 'luxon'
import { BaseModel, column } from '@adonisjs/lucid/orm'

export default class Student extends BaseModel {
  @column({ isPrimary: true })
  declare id: number

  @column()
  declare name: string

  @column()
  declare email: string

  @column()
  declare phone: string | null

  @column()
  declare department: string

  @column()
  declare semester: number

  @column.dateTime({ autoCreate: true })
  declare createdAt: DateTime

  @column.dateTime({ autoCreate: true, autoUpdate: true })
  declare updatedAt: DateTime
}
```

Remember:

```
students table -> Student model
```

---

## 6. Create Student Validator

Validator checks form data before saving it into the database.

Create validator:

```
node ace make:validator student
```

Open:

```
app/validators/student.ts
```

Paste this code:

```ts
import vine from '@vinejs/vine'

export const createStudentValidator = vine.create({
  name: vine.string().trim().minLength(3),
  email: vine.string().email(),
  phone: vine.string().optional(),
  department: vine.string().trim().minLength(2),
  semester: vine.number().min(1).max(8),
})
```

Meaning:

```
name       = required string, minimum 3 characters
email      = required valid email
phone      = optional string
department = required string, minimum 2 characters
semester   = required number from 1 to 8
```

This is the simple syntax. The important thing is that the controller imports `createStudentValidator` and uses `request.validateUsing()`.

---

## 7. Add Fake Data with Factory and Seeder

Fake data is useful for demo and testing. You do not need to type 20 students manually.

Simple idea:

```
Factory -> makes fake student data
Seeder  -> inserts fake student data into database
Command -> runs the seeder
```

### 7.1 Create Student Factory

Run this command:

```
node ace make:factory Student
```

A file will be created:

```
database/factories/student_factory.ts
```

Open that file and paste this code:

```ts
import Student from '#models/student'
import { Factory } from '@adonisjs/lucid/factories'

export const StudentFactory = Factory.define(Student, ({ faker }) => {
  return {
```

```
    name: faker.person.fullName(),
    email: faker.string.uuid() + '@example.com',
    phone: faker.phone.number(),
    department: faker.helpers.arrayElement(['CSE', 'EEE', 'BBA', 'English']),
    semester: faker.number.int({ min: 1, max: 8 }),
  }
}).build()
```

Meaning:

```
faker.person.fullName()       = fake name
faker.string.uuid()           = unique fake email part
faker.phone.number()          = fake phone number
arrayElement([...])           = random department from list
faker.number.int({ min, max }) = random semester between 1 and 8
```

### 7.2 Create Student Seeder

Run this command:

```
node ace make:seeder Student
```

A file will be created:

```
database/seeders/student_seeder.ts
```

Open that file and paste this code:

```
import { BaseSeeder } from '@adonisjs/lucid/seeders'
import { StudentFactory } from '#database/factories/student_factory'

export default class extends BaseSeeder {
  async run() {
    await StudentFactory.createMany(20)
  }
}
```

Meaning:

```
StudentFactory.createMany(20) = create and insert 20 fake students
```

### 7.3 Run Seeder

Run this command:

```
node ace db:seed
```

Now open:

```
http://localhost:3333/students
```

You should see fake students in the table.

### 7.4 If duplicate email error happens

Use this email line in factory:

```
email: faker.string.uuid() + '@example.com',
```

This makes email unique.

Remember:

```
Factory = fake data design
Seeder  = fake data insert
```

---

## 8. Create Web Controller

Controller handles the application logic.

```
node ace make:controller students_web
```

Open:

```
app/controllers/students_web_controller.ts
```

Paste this code:

```
import type { HttpContext } from '@adonisjs/core/http'
import Student from '#models/student'
import { createStudentValidator } from '#validators/student'

export default class StudentsWebController {
  async index({ view }: HttpContext) {
    const students = await Student.all()

    return view.render('students/index', {
      students,
    })
  }

  async store({ request, response }: HttpContext) {
    const payload = await request.validateUsing(createStudentValidator)

    await Student.create(payload)

    return response.redirect().toPath('/students')
  }

  async edit({ params, view }: HttpContext) {
    const student = await Student.findOrFail(params.id)
```

```
    return view.render('students/edit', {
      student,
    })
  }

  async update({ params, request, response }: HttpContext) {
    const student = await Student.findOrFail(params.id)

    const payload = await request.validateUsing(createStudentValidator)

    student.merge(payload)
    await student.save()

    return response.redirect().toPath('/students')
  }

  async destroy({ params, response }: HttpContext) {
    const student = await Student.findOrFail(params.id)

    await student.delete()

    return response.redirect().toPath('/students')
  }
}
```

Controller method meanings:

```
index   = show all students
store   = validate and add new student
edit    = show edit form
update  = validate and update student data
destroy = delete student
```

---

## 9. Add Routes

Open:

`start/routes.ts`

Paste this code:

```
import router from '@adonisjs/core/services/router'

const StudentsWebController = () => import('#controllers/students_web_controller')

router.get('/students', [StudentsWebController, 'index'])
router.post('/students', [StudentsWebController, 'store'])
```

```
router.get('/students/:id/edit', [StudentsWebController, 'edit'])
router.post('/students/:id/update', [StudentsWebController, 'update'])
router.post('/students/:id/delete', [StudentsWebController, 'destroy'])
```

Route meanings:

```
GET  /students              -> show all students
POST /students              -> add new student
GET  /students/:id/edit     -> show edit form
POST /students/:id/update   -> update student
POST /students/:id/delete   -> delete student
```

Why lazy import?

```
const StudentsWebController = () => import('#controllers/students_web_controller')
```

This avoids controller import errors in some AdonisJS setups.

---

## 10. Create Edge View Folder

```
mkdir -p resources/views/students
```

---

## 11. Create `index.edge`

Create file:

`resources/views/students/index.edge`

Paste this code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Management</title>

  <style>
    body {
      font-family: Arial;
      background: #f1f5f9;
      padding: 30px;
    }

    .box {
      background: white;
      padding: 20px;
      border-radius: 10px;
```

```
  margin-bottom: 20px;
}

input {
  padding: 10px;
  margin: 5px;
  width: 180px;
}

button, a {
  padding: 8px 12px;
  text-decoration: none;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

.add {
  background: #2563eb;
  color: white;
}

.edit {
  background: #facc15;
  color: black;
}

.delete {
  background: #dc2626;
  color: white;
}

table {
  width: 100%;
  border-collapse: collapse;
  background: white;
}

th, td {
  border: 1px solid #ddd;
  padding: 10px;
  text-align: left;
}

th {
  background: #0f172a;
```

```
      color: white;
    }
  </style>
</head>

<body>
  <h1>Student Management System</h1>

  <div class="box">
    <h2>Add Student</h2>

    <form method="POST" action="/students">
      <input type="text" name="name" placeholder="Name" required>
      <input type="email" name="email" placeholder="Email" required>
      <input type="text" name="phone" placeholder="Phone">
      <input type="text" name="department" placeholder="Department" required>
      <input type="number" name="semester" placeholder="Semester" required>

      <button class="add" type="submit">Add Student</button>
    </form>
  </div>

  <div class="box">
    <h2>All Students</h2>

    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Email</th>
          <th>Phone</th>
          <th>Department</th>
          <th>Semester</th>
          <th>Action</th>
        </tr>
      </thead>

      <tbody>
        @each(student in students)
          <tr>
            <td>{{ student.id }}</td>
            <td>{{ student.name }}</td>
            <td>{{ student.email }}</td>
            <td>{{ student.phone }}</td>
            <td>{{ student.department }}</td>
```

```
            <td>{{ student.semester }}</td>

            <td>
              <a class="edit" href="/students/{{ student.id }}/edit">Edit</a>

              <form method="POST"
                    action="/students/{{ student.id }}/delete"
                    style="display:inline;">
                <button class="delete" type="submit">Delete</button>
              </form>
            </td>
          </tr>
        @else
          <tr>
            <td colspan="7">No students found</td>
          </tr>
        @end
      </tbody>
    </table>
  </div>
</body>
</html>
```

---

## 12. Create `edit.edge`

Create file:

`resources/views/students/edit.edge`

Paste this code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Edit Student</title>

  <style>
    body {
      font-family: Arial;
      background: #f1f5f9;
      padding: 30px;
    }

    .box {
      background: white;
      padding: 20px;
```

```
      border-radius: 10px;
      width: 500px;
    }

    input {
      display: block;
      padding: 10px;
      margin: 10px 0;
      width: 95%;
    }

    button, a {
      padding: 10px 14px;
      border: none;
      border-radius: 5px;
      text-decoration: none;
    }

    button {
      background: #16a34a;
      color: white;
    }

    a {
      background: #64748b;
      color: white;
    }
  </style>
</head>

<body>
  <div class="box">
    <h1>Edit Student</h1>

    <form method="POST" action="/students/{{ student.id }}/update">
      <input type="text" name="name" value="{{ student.name }}" required>
      <input type="email" name="email" value="{{ student.email }}" required>
      <input type="text" name="phone" value="{{ student.phone }}">
      <input type="text" name="department" value="{{ student.department }}" required>
      <input type="number" name="semester" value="{{ student.semester }}" required>

      <button type="submit">Update Student</button>
      <a href="/students">Back</a>
    </form>
  </div>
</body>
```

```
</html>
```

---

## 13. Run the Project

```
npm run dev
```

Open browser:

```
http://localhost:3333/students
```

Now you can:

```
Add student
Show all students
Edit student
Update student
Delete student
Validate input
Add fake data
```

---

## 14. Full Terminal Command Summary

Use this summary when starting from zero:

```
npm create adonisjs@latest student-api
cd student-api

node ace add edge

node ace make:migration students
# edit migration file before running the next command
node ace migration:run

node ace make:model Student
node ace make:validator student

node ace make:factory Student
node ace make:seeder Student
node ace db:seed

node ace make:controller students_web

mkdir -p resources/views/students

npm run dev
```

If migration mistake happens:

```
node ace migration:rollback
node ace migration:run
```

---

## 15. Common Errors and Fixes

### Error 1: Cannot find controller module

Error example:

```
Cannot find module app/controllers/students_web_controller.js
```

Fix:

```
ls app/controllers
```

File must be:

```
students_web_controller.ts
```

Routes should use lazy import:

```
const StudentsWebController = () => import('#controllers/students_web_controller')
```

Do not write `.js` or `.ts` at the end.

### Error 2: Migration command not found

Fix:

```
node ace add lucid
```

Then try again:

```
node ace make:migration students
```

### Error 3: Form submit gives CSRF error

If you see `E_BAD_CSRF_TOKEN` or `403`, add this inside every POST form:

```
{{ csrfField() }}
```

Example:

```
<form method="POST" action="/students">
  {{ csrfField() }}
  <input type="text" name="name" placeholder="Name" required>
  <button type="submit">Add Student</button>
</form>
```

If `csrfField()` itself gives error, remove it. Your project may not have CSRF protection enabled.

**Error 4: Validator import error**

If this line gives error:

```
import { createStudentValidator } from '#validators/student'
```

Check file exists:

```
ls app/validators
```

File must be:

```
student.ts
```

**Error 5: Validation error on semester**

If semester gives validation error, make sure the form input is number:

```
<input type="number" name="semester" required>
```

And send value between 1 and 8.

**Error 6: Factory import error**

If this line gives error:

```
import { StudentFactory } from '#database/factories/student_factory'
```

Check the file exists:

```
ls database/factories
```

File must be:

```
student_factory.ts
```

**Error 7: `Factory` import error**

If this line gives error:

```
import { Factory } from '@adonisjs/lucid/factories'
```

Use this alternative factory import:

```
import factory from '@adonisjs/lucid/factories'
```

Then change this:

```
export const StudentFactory = Factory.define(Student, ({ faker }) => {
```

to this:

```
export const StudentFactory = factory.define(Student, ({ faker }) => {
```

---

## 16. Exam Explanation

Say this:

```
This is a Student Management System built with AdonisJS.
I used Lucid ORM for database operations.
Migration creates the students table.
Model connects the code with the students table.
Validator checks the input data before saving.
Factory creates fake student data.
Seeder inserts fake data into the database.
Controller handles the application logic.
Edge template renders the HTML user interface.
Routes connect URLs with controller methods.
```

Short formula:

```
Migration -> Model -> Validator -> Factory/Seeder -> Controller -> Route -> Edge View
```

Controller method formula:

```
index   = list
store   = validate and add
edit    = edit form
update  = validate and update
destroy = delete
```

---

## 17. Official References

- AdonisJS Installation: https://docs.adonisjs.com/installation
- EdgeJS Frontend: https://docs.adonisjs.com/guides/frontend/edgejs
- Lucid ORM: https://docs.adonisjs.com/guides/database/lucid
- Routing: https://docs.adonisjs.com/guides/basics/routing
- Controllers: https://docs.adonisjs.com/guides/basics/controllers